

LAB Assignment No 4

Interfacing LED with ESP32 and Web Dashboard Control

Objective:

- Learn how to control an LED connected to ESP32 using a web-based dashboard.
- Understand communication between ESP32 and Firebase Realtime Database.
- Observe real-time LED control via web interface.

Materials Required:

1. ESP32 (simulated in Wokwi)
2. LED
3. 220Ω resistor
4. Breadboard and jumper wires
5. Computer with internet access (for Wokwi simulation and Firebase)
6. Web browser

Lab Procedure

Step 1: Setup Wokwi ESP32 Simulation

1. Open Wokwi online simulator.
2. Add an ESP32 module.
3. Connect an LED to **GPIO 2** of ESP32 using a 220Ω resistor (long leg to GPIO, short leg to GND).
4. Ensure wiring matches the schematic:
 - ESP32 Pin 2 → Resistor → LED Anode (+)
 - LED Cathode (-) → GND

Step 2: Configure Firebase Realtime Database

1. Go to [Firebase Console](#).
2. Create a new project (if not already created).
3. Go to **Realtime Database** → **Create Database**.
4. Set rules to **public for testing** (students can change later for security):

```
</> JSON

{
  "rules": {
    ".read": true,
    ".write": true
  }
}
```

5. Copy the database URL (it looks like `https://your-project-id.firebaseio.com/led.json`).
6. Replace the URL in `app.js` and `ESP32 sketch` with your database URL.

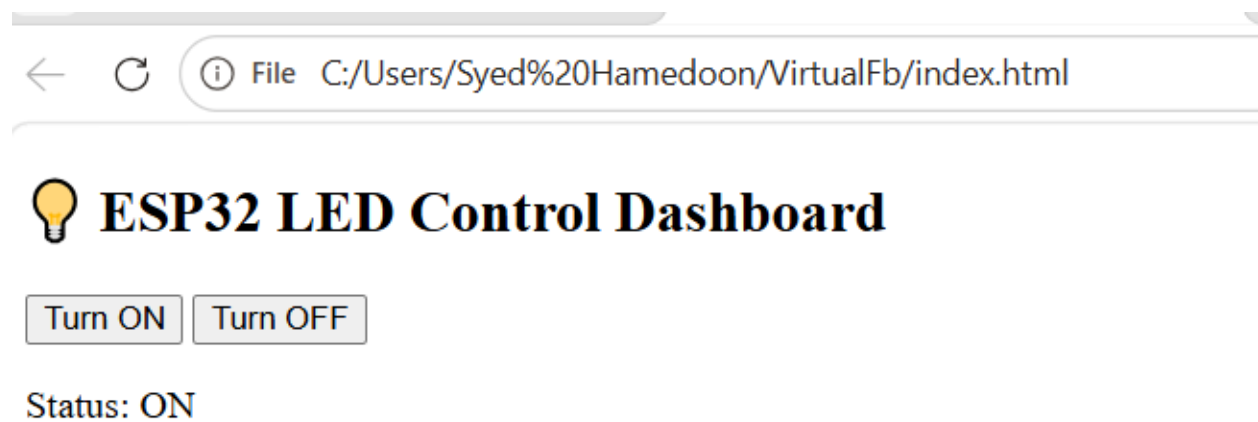
The screenshot shows the Firebase Realtime Database console for a project named 'VirtualMCU'. The left sidebar contains the Firebase logo, search bar, and navigation links for Project Overview, Settings, and Realtime Database. The main content area displays the 'Realtime Database' page with tabs for Data, Rules, Backups, Usage, and Extensions. A warning message states: 'Your security rules are defined as public, so anyone can steal, modify, or delete data in your database'. Below this, the database URL is shown as 'https://virtualmcu-2ee29-default-rtdb.firebaseio.com/'. A tree view shows a node named 'led' with a 'state' of 1 and an 'updatedAt' timestamp of '4/4/2026, 12:39:35 PM'.

Step 3: Prepare the Web Dashboard

1. Create a folder on your computer called `ESP32_LED_Dashboard`.
2. Inside, create two files:
 - `index.html`
 - `app.js`
3. Paste the provided code:

`index.html`

My web-based dash board



`index.html`

```
<!DOCTYPE html>
<html>
<head>
  <title>ESP32 LED Control</title>
</head>
<body>

  <h2>💡 ESP32 LED Control Dashboard</h2>

  <button onclick="turnOn()">Turn ON</button>
  <button onclick="turnOff()">Turn OFF</button>

  <p>Status: <span id="status">Unknown</span></p>

  <!-- Link JS file -->
  <script src="app.js"></script>

</body>
```

```
</html>
```

app.js

```
const baseURL = "https://virtualmcu-2ee29-default-rtdb.firebaseio.com";

function updateLED(state) {
  fetch(`${baseURL}/led.json`, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      state: state,
      updatedAt: new Date().toLocaleString()
    })
  })
  .then(res => res.json())
  .then(data => {
    console.log("Updated:", data);
    document.getElementById("status").innerText = state === 1 ? "ON" : "OFF";
  })
  .catch(err => console.error(err));
}

function turnOn() {
  updateLED(1);
}

function turnOff() {
  updateLED(0);
}
```

Step 4: Program ESP32

1. Open Arduino IDE or Wokwi code editor.
2. Paste the provided `Sketch.ino` code:

Sketch.ino

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
```

```

const char* ssid = "Wokwi-GUEST";
const char* password = "";

// 🛠 IMPORTANT: point to specific key
String firebaseURL = "https://virtualmcu-2ee29-default-rtdb.firebaseio.com/led.json";

int ledPin = 2;

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);

  WiFi.begin(ssid, password);
  Serial.print("Connecting...");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("Connected!");
}

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;

    http.begin(firebaseURL);
    int httpStatusCode = http.GET();

    if (httpStatusCode > 0) {
      String payload = http.getString();

      Serial.print("Firebase Data: ");
      Serial.println(payload);

      // ✅ Parse JSON properly
      StaticJsonDocument<200> doc;
      DeserializationError error = deserializeJson(doc, payload);

      if (!error) {
        int state = doc["state"]; // 🛠 get value

        Serial.print("Parsed State: ");
        Serial.println(state);
      }
    }
  }
}

```

```
    if (state == 1) {  
        digitalWrite(ledPin, HIGH);  
        Serial.println("LED ON");  
    } else {  
        digitalWrite(ledPin, LOW);  
        Serial.println("LED OFF");  
    }  
  
    } else {  
        Serial.print("JSON Error: ");  
        Serial.println(error.c_str());  
    }  
  
    } else {  
        Serial.println("Error in HTTP request");  
    }  
  
    http.end();  
}  
  
delay(1000);  
}
```

Step 1: Add Library in Wokwi

In Wokwi:

 Add to `libraries.txt` :

ArduinoJson

Step 2: Replace Your Loop Code

 C++

```
#include <ArduinoJson.h>
```

WOKWI window

WOKWI

SAVE

SHARE

testnew

sketch.ino diagram.json Library Manager libraries.txt

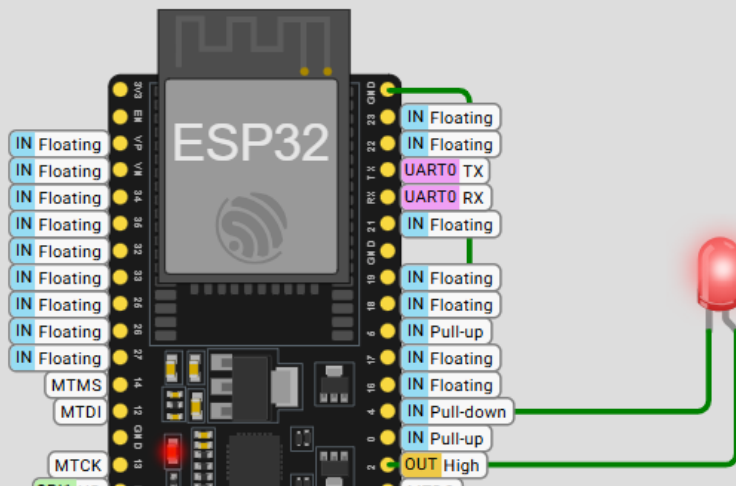
```
1  #include <WiFi.h>
2  #include <HTTPClient.h>
3  #include <ArduinoJson.h>
4
5  const char* ssid = "Wokwi-GUEST";
6  const char* password = "";
7
8  // 🔥 IMPORTANT: point to specific key
9  String firebaseURL = "https://virtualmcu-2ee29-default-rtdb.firebaseio.com/led.";
10
11  int ledPin = 2;
12
13  void setup() {
14      Serial.begin(115200);
15      pinMode(ledPin, OUTPUT);
16
17      WiFi.begin(ssid, password);
18      Serial.print("Connecting...");
19  }
```

My WOKWI simulation Window

Connect LED anode with pin2 of EPS32 and cathode of LED with Ground and make circuit diagram accordingly. After save the code press the run button it take some times to connect then after connecting go to web based dash board.

Simulation

00:21



ESP32

IN Floating

IN Floating

IN Floating

IN Floating

IN Floating

IN Floating

IN Floating

IN Floating

MTMS

MTDI

MTCK

SPI1 HD

IN Floating

IN Floating

UART0 TX

UART0 RX

IN Floating

IN Floating

IN Floating

IN Floating

IN Pull-up

IN Floating

IN Pull-down

IN Pull-up

OUT High

MTDO

Connecting.....Connected!

Firestore Data: {"state":1,"updatedAt":"4/4/2026, 12:39:35 PM"}

Parsed State: 1

LED ON

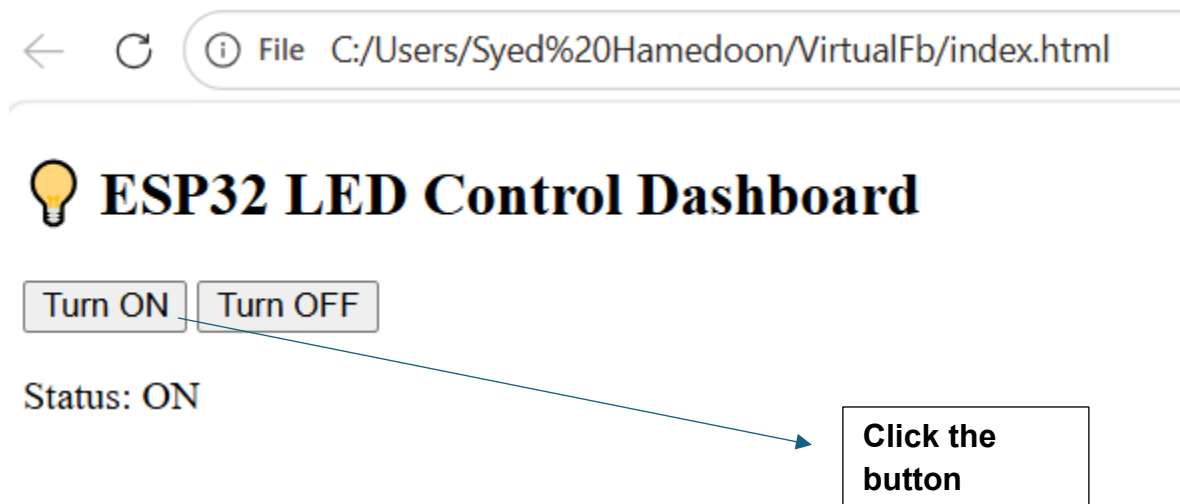
Firestore Data: {"state":1,"updatedAt":"4/4/2026, 12:39:35 PM"}

Parsed State: 1

LED ON

Step 5: Test the Web Dashboard

1. Click **Turn ON** button → LED in simulation should turn on.
2. Click **Turn OFF** button → LED should turn off.
3. Observe **status text** on the dashboard updates to reflect the LED state.
4. Check **Serial Monitor** → it will print Firebase data, parsed state, and LED actions.



Observe the output on wokwi simulation diagram

Simulation

Connecting.....Connected!

```

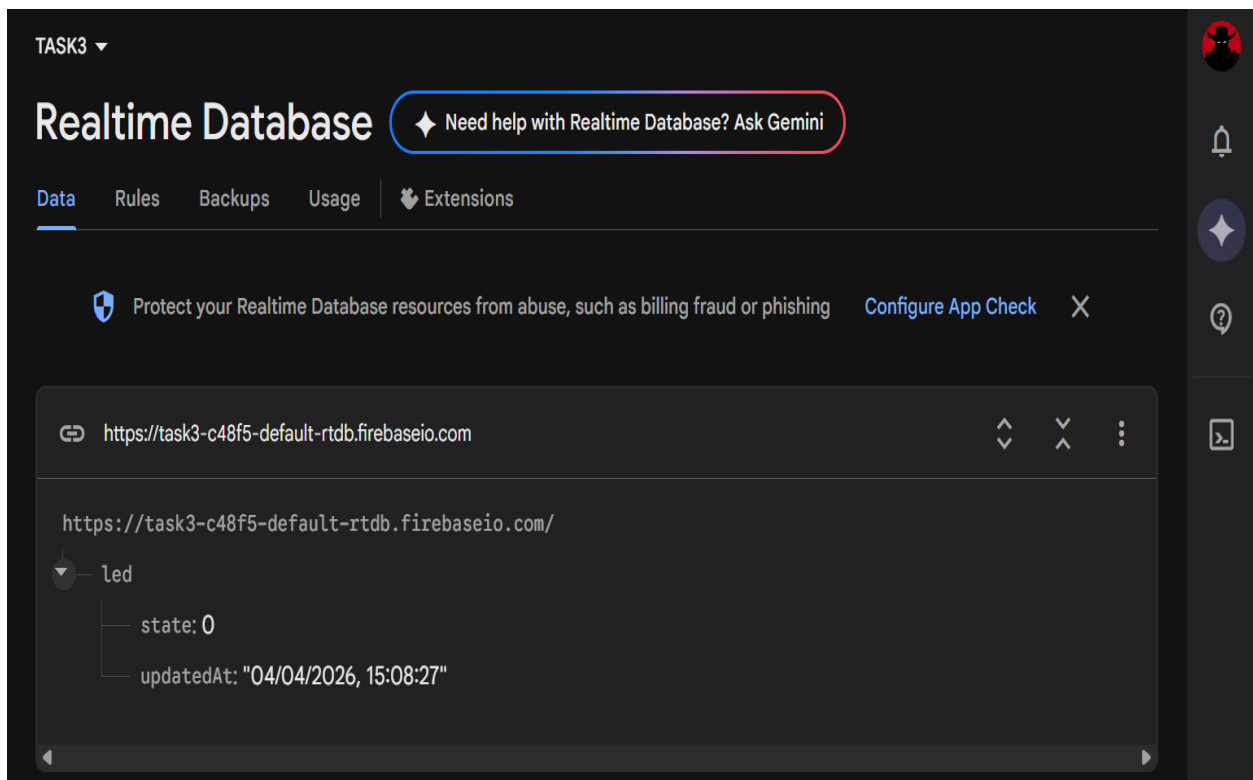
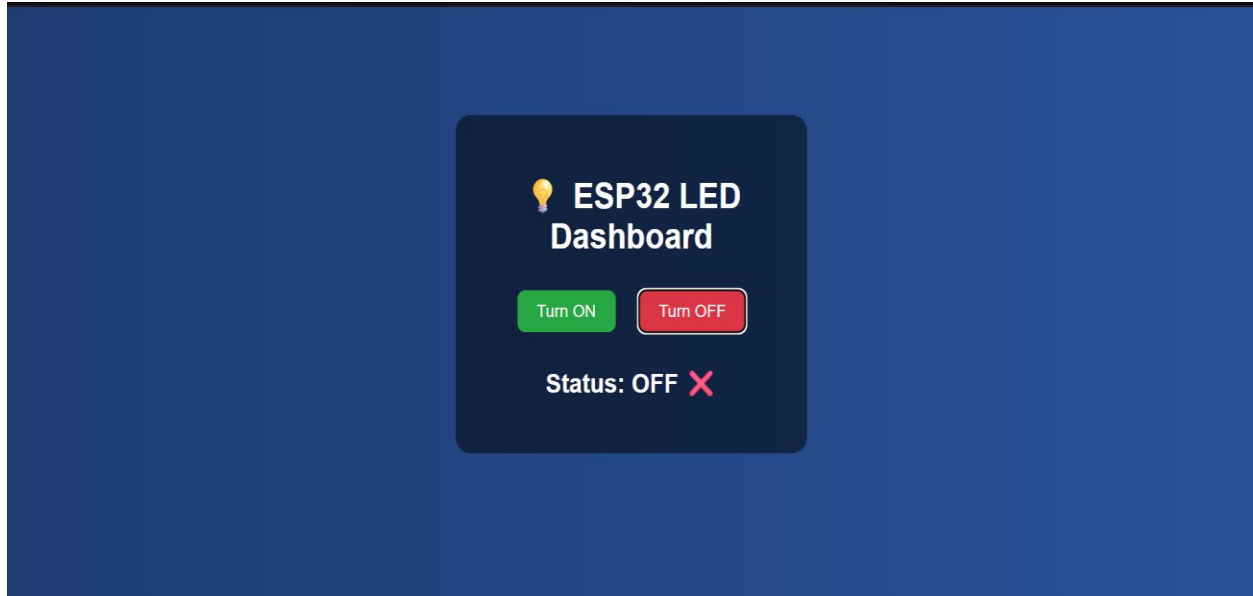
Firebase Data: {"state":1,"updatedAt":"4/4/2026, 12:39:35 PM"}
Parsed State: 1
LED ON
Firebase Data: {"state":1,"updatedAt":"4/4/2026, 12:39:35 PM"}
Parsed State: 1
LED ON
    
```

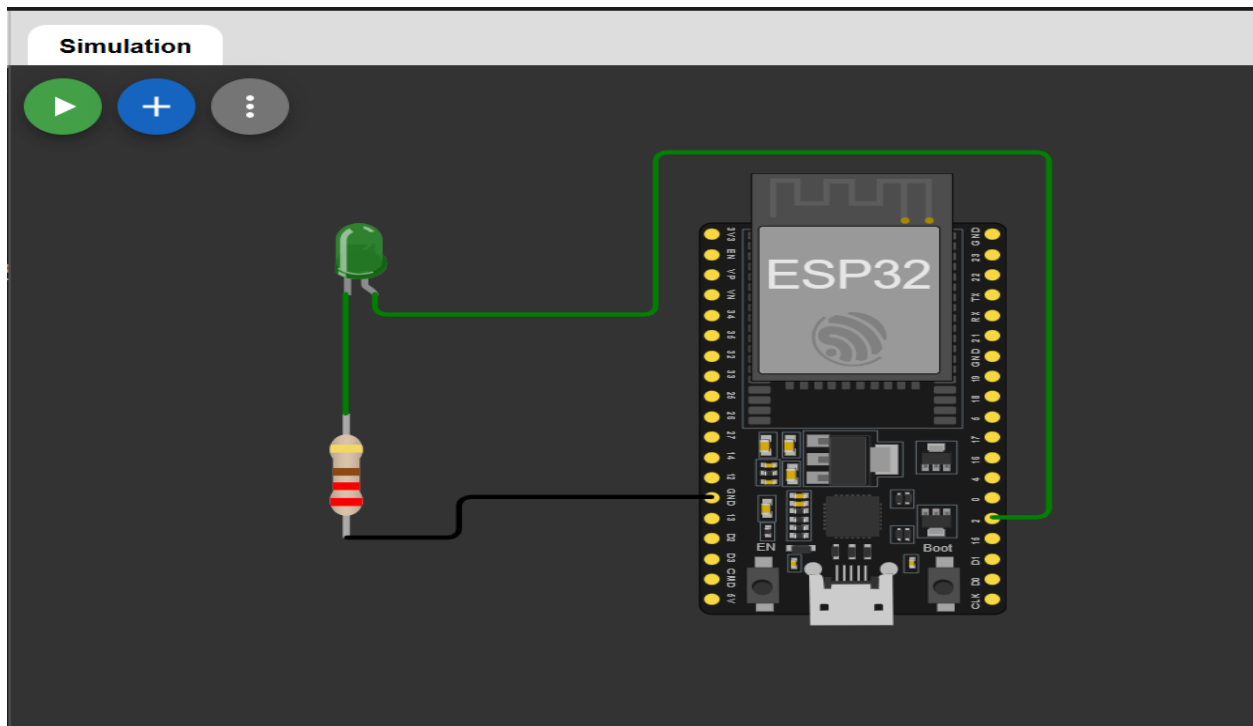
Expected Serial Output

```

Firebase Data: {"state":1,"updatedAt":"..."}
Parsed State: 1
LED ON
    
```

SOL:





```
1  #include <WiFi.h>
2  #include <HTTPClient.h>
3  #include <ArduinoJson.h>
4
5  const char* ssid = "Wokwi-GUEST";
6  const char* password = "";
7
8  String firebaseURL = "https://task3-c48f5-default-rtdb.firebaseio.com/le
9
10 int ledPin = 2;
11
12 void setup() {
13     Serial.begin(115200);
14     pinMode(ledPin, OUTPUT);
15
16     WiFi.begin(ssid, password);
17     Serial.print("Connecting...");
18
19     while (WiFi.status() != WL_CONNECTED) {
20         delay(500);
21         Serial.print(".");
22     }
23
24     Serial.println("\nConnected!");
25 }
```

```

27 void loop() {
28   if (WiFi.status() == WL_CONNECTED) {
29     HTTPClient http;
30
31     http.begin(firebaseURL);
32     int httpStatusCode = http.GET();
33
34     if (httpStatusCode > 0) {
35       String payload = http.getString();
36
37       StaticJsonDocument<200> doc;
38       DeserializationError error = deserializeJson(doc, payload);
39
40       if (!error) {
41         int state = doc["state"];
42
43         if (state == 1) {
44           digitalWrite(ledPin, HIGH);
45           Serial.println("LED ON");
46         } else {
47           digitalWrite(ledPin, LOW);
48           Serial.println("LED OFF");
49         }
50       }
51     }

```



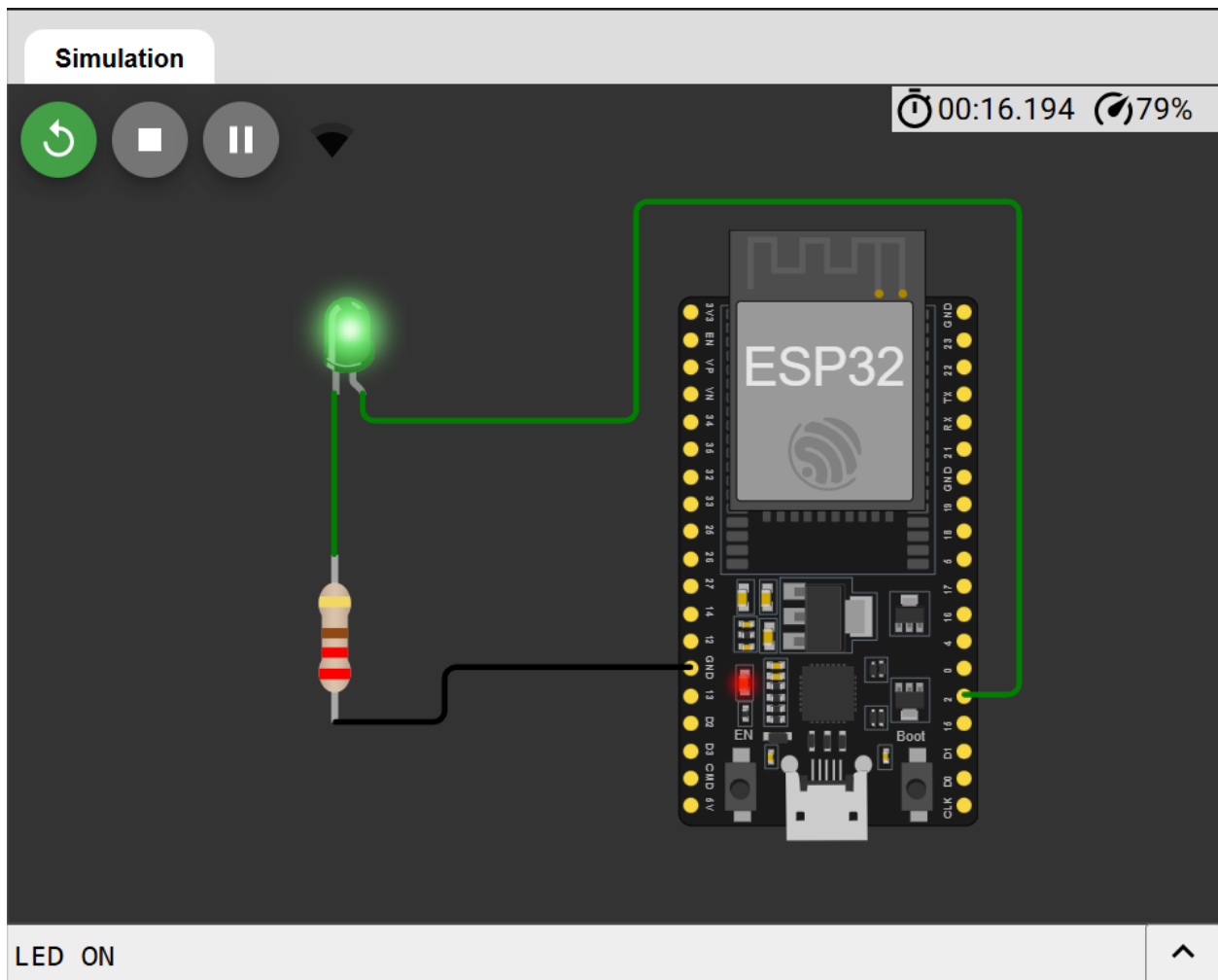
ESP32 LED Dashboard

Turn ON

Turn OFF

Status: ON





```
entry 0x400805dc
Connecting.....
Connected!
LED ON
LED ON
LED ON
LED ON
```

